

BCC202 - Estruturas de Dados I

Aula 20: Ordenação em Tempo Linear

Pedro Silva

Universidade Federal de Ouro Preto, UFOP
Departamento de Computação, DECOM
Email: silvap@ufop.edu.br



Conteúdo

Introdução

Counting Sort

Radix Sort

Bucket Sort

Considerações Finais

Bibliografia

Introdução

Ordenação em tempo linear

Algoritmos de ordenação por **comparação**

- ▶ *Insert Sort.*
- ▶ *Select Sort.*
- ▶ *Quick Sort.*
- ▶ *Merge Sort.*
- ▶ *Heap Sort...*

Possuem **limite assintótico**: $O(n \lg n)$.

Podem existir algoritmos melhores?

Ordenação em tempo linear

Algoritmos de ordenação por **comparação**

- ▶ *Insert Sort.*
- ▶ *Select Sort.*
- ▶ *Quick Sort.*
- ▶ *Merge Sort.*
- ▶ *Heap Sort...*

Possuem **limite assintótico**: $O(n \lg n)$.

Podem existir algoritmos melhores?

Ordenação em tempo linear

A resposta é **SIM**, desde que:

- ▶ A entrada possui características especiais.
- ▶ Algumas restrições devem ser respeitadas.
- ▶ O algoritmo não é baseado puramente em comparações.
- ▶ A implementação deve ser feita de maneira adequada.

Tempo linear: $\Theta(n)$.

Algoritmos de ordenação em tempo linear

Algoritmos:

- ▶ Ordenação por contagem (*Counting Sort*).
- ▶ *Radix Sort*.
- ▶ *Bucket Sort*.

Counting Sort

Definição

Pressupõe que cada elemento da entrada é um inteiro na faixa de 0 a k , para algum inteiro k .

Ideia básica:

- ▶ Determinar para cada elemento da entrada x o número de elementos maiores que x .
- ▶ Com esta informação, determinar a posição de cada elemento.
 - ▶ Ex.: Se 17 elementos forem menores que x então x ocupa a posição de saída 18.

Algoritmo:

- ▶ Assumimos que o vetor de entrada é $A[1, \dots, n]$;
- ▶ Outros dois vetores são utilizados:
 - ▶ $B[1, \dots, n]$ – armazena a saída ordenada;
 - ▶ $C[0, \dots, k]$ – é utilizado para armazenamento temporário.

1 Algorithm: COUNTING_SORT**Input:** int* A, int n, int k**2 begin****3 for** $i \leftarrow 0$ **to** k **do****4 |** $C[i] \leftarrow 0$ **5 end****6 for** $j \leftarrow 1$ **to** n **do****7 |** $C[A[j]] \leftarrow C[A[j]] + 1$ **8 end****9 for** $i \leftarrow 1$ **to** k **do****10 |** $C[i] \leftarrow C[i] + C[i - 1]$ **11 end****12 for** $j \leftarrow n$ **to** 1 **do****13 |** $B[C[A[j]]] \leftarrow A[j]$ **14 |** $C[A[j]] \leftarrow C[A[j]] - 1$ **15 end****16 end**

A →

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

k = 5

C →

0	1	2	3	4	5

B →

1	2	3	4	5	6	7	8

1 Algorithm: COUNTING_SORT

Input: int* A, int n, int k**2 begin****3 for** $i \leftarrow 0$ **to** k **do****4 |** $C[i] \leftarrow 0$ **5 end****6 for** $j \leftarrow 1$ **to** n **do****7 |** $C[A[j]] \leftarrow C[A[j]] + 1$ **8 end****9 for** $i \leftarrow 1$ **to** k **do****10 |** $C[i] \leftarrow C[i] + C[i - 1]$ **11 end****12 for** $j \leftarrow n$ **to** 1 **do****13 |** $B[C[A[j]]] \leftarrow A[j]$ **14 |** $C[A[j]] \leftarrow C[A[j]] - 1$ **15 end****16 end**

A →

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

 $k = 5$

C →

0	1	2	3	4	5
0	0	0	0	0	0

B →

1	2	3	4	5	6	7	8

1 Algorithm: COUNTING_SORT

Input: int* A, int n, int k**2 begin****3 for** $i \leftarrow 0$ **to** k **do****4 |** $C[i] \leftarrow 0$ **5 end****6 for** $j \leftarrow 1$ **to** n **do****7 |** $C[A[j]] \leftarrow C[A[j]] + 1$ **8 end****9 for** $i \leftarrow 1$ **to** k **do****10 |** $C[i] \leftarrow C[i] + C[i - 1]$ **11 end****12 for** $j \leftarrow n$ **to** 1 **do****13 |** $B[C[A[j]]] \leftarrow A[j]$ **14 |** $C[A[j]] \leftarrow C[A[j]] - 1$ **15 end****16 end**

A →

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

 $k = 5$

C →

0	1	2	3	4	5
2	0	2	3	0	1

B →

1	2	3	4	5	6	7	8

1 Algorithm: COUNTING_SORT

Input: int* A, int n, int k**2 begin****3 for** $i \leftarrow 0$ **to** k **do****4 |** $C[i] \leftarrow 0$ **5 end****6 for** $j \leftarrow 1$ **to** n **do****7 |** $C[A[j]] \leftarrow C[A[j]] + 1$ **8 end****9 for** $i \leftarrow 1$ **to** k **do****10 |** $C[i] \leftarrow C[i] + C[i - 1]$ **11 end****12 for** $j \leftarrow n$ **to** 1 **do****13 |** $B[C[A[j]]] \leftarrow A[j]$ **14 |** $C[A[j]] \leftarrow C[A[j]] - 1$ **15 end****16 end**

A →

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

 $k = 5$

C →

0	1	2	3	4	5
2	2	4	7	7	8

B →

1	2	3	4	5	6	7	8

1 Algorithm: COUNTING_SORT

Input: int* A, int n, int k**2 begin****3 for** $i \leftarrow 0$ **to** k **do** | $C[i] \leftarrow 0$ **5 end****6 for** $j \leftarrow 1$ **to** n **do** | $C[A[j]] \leftarrow C[A[j]] + 1$ **8 end****9 for** $i \leftarrow 1$ **to** k **do** | $C[i] \leftarrow C[i] + C[i - 1]$ **11 end****12 for** $j \leftarrow n$ **to** 1 **do** | $B[C[A[j]]] \leftarrow A[j]$ | $C[A[j]] \leftarrow C[A[j]] - 1$ **15 end****16 end**

A →

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

 $k = 5$

C →

0	1	2	3	4	5
2	2	4	6	7	8

B →

1	2	3	4	5	6	7	8
						3	

1 Algorithm: COUNTING_SORT

Input: int* A, int n, int k**2 begin****3 for** $i \leftarrow 0$ **to** k **do****4 |** $C[i] \leftarrow 0$ **5 end****6 for** $j \leftarrow 1$ **to** n **do****7 |** $C[A[j]] \leftarrow C[A[j]] + 1$ **8 end****9 for** $i \leftarrow 1$ **to** k **do****10 |** $C[i] \leftarrow C[i] + C[i - 1]$ **11 end****12 for** $j \leftarrow n$ **to** 1 **do****13 |** $B[C[A[j]]] \leftarrow A[j]$ **14 |** $C[A[j]] \leftarrow C[A[j]] - 1$ **15 end****16 end**

A →

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

 $k = 5$

C →

0	1	2	3	4	5
1	2	4	6	7	8

B →

1	2	3	4	5	6	7	8
	0					3	

1 Algorithm: COUNTING_SORT

Input: int* A, int n, int k**2 begin****3 for** $i \leftarrow 0$ **to** k **do****4 |** $C[i] \leftarrow 0$ **5 end****6 for** $j \leftarrow 1$ **to** n **do****7 |** $C[A[j]] \leftarrow C[A[j]] + 1$ **8 end****9 for** $i \leftarrow 1$ **to** k **do****10 |** $C[i] \leftarrow C[i] + C[i - 1]$ **11 end****12 for** $j \leftarrow n$ **to** 1 **do****13 |** $B[C[A[j]]] \leftarrow A[j]$ **14 |** $C[A[j]] \leftarrow C[A[j]] - 1$ **15 end****16 end**

A →

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

 $k = 5$

C →

0	1	2	3	4	5
1	2	4	5	7	8

B →

1	2	3	4	5	6	7	8
	0				3	3	

1 Algorithm: COUNTING_SORT

Input: int* A, int n, int k**2 begin****3 for** $i \leftarrow 0$ **to** k **do****4 |** $C[i] \leftarrow 0$ **5 end****6 for** $j \leftarrow 1$ **to** n **do****7 |** $C[A[j]] \leftarrow C[A[j]] + 1$ **8 end****9 for** $i \leftarrow 1$ **to** k **do****10 |** $C[i] \leftarrow C[i] + C[i - 1]$ **11 end****12 for** $j \leftarrow n$ **to** 1 **do****13 |** $B[C[A[j]]] \leftarrow A[j]$ **14 |** $C[A[j]] \leftarrow C[A[j]] - 1$ **15 end****16 end**

	1	2	3	4	5	6	7	8
A →	2	5	3	0	2	3	0	3

 $k = 5$

	0	1	2	3	4	5
C →	1	2	3	5	7	8

	1	2	3	4	5	6	7	8
B →		0		2		3	3	

1 Algorithm: COUNTING_SORT

Input: int* A, int n, int k**2 begin****3 for** $i \leftarrow 0$ **to** k **do****4 |** $C[i] \leftarrow 0$ **5 end****6 for** $j \leftarrow 1$ **to** n **do****7 |** $C[A[j]] \leftarrow C[A[j]] + 1$ **8 end****9 for** $i \leftarrow 1$ **to** k **do****10 |** $C[i] \leftarrow C[i] + C[i - 1]$ **11 end****12 for** $j \leftarrow n$ **to** 1 **do****13 |** $B[C[A[j]]] \leftarrow A[j]$ **14 |** $C[A[j]] \leftarrow C[A[j]] - 1$ **15 end****16 end**

A →

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

 $k = 5$

C →

0	1	2	3	4	5
0	2	3	5	7	8

B →

1	2	3	4	5	6	7	8
0	0		2		3	3	

1 Algorithm: COUNTING_SORT

Input: int* A, int n, int k**2 begin****3 for** $i \leftarrow 0$ **to** k **do****4 |** $C[i] \leftarrow 0$ **5 end****6 for** $j \leftarrow 1$ **to** n **do****7 |** $C[A[j]] \leftarrow C[A[j]] + 1$ **8 end****9 for** $i \leftarrow 1$ **to** k **do****10 |** $C[i] \leftarrow C[i] + C[i - 1]$ **11 end****12 for** $j \leftarrow n$ **to** 1 **do****13 |** $B[C[A[j]]] \leftarrow A[j]$ **14 |** $C[A[j]] \leftarrow C[A[j]] - 1$ **15 end****16 end**

A →

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

 $k = 5$

C →

0	1	2	3	4	5
0	2	3	4	7	8

B →

1	2	3	4	5	6	7	8
0	0		2	3	3	3	

1 Algorithm: COUNTING_SORT

Input: int* A, int n, int k**2 begin****3 for** $i \leftarrow 0$ **to** k **do****4 |** $C[i] \leftarrow 0$ **5 end****6 for** $j \leftarrow 1$ **to** n **do****7 |** $C[A[j]] \leftarrow C[A[j]] + 1$ **8 end****9 for** $i \leftarrow 1$ **to** k **do****10 |** $C[i] \leftarrow C[i] + C[i - 1]$ **11 end****12 for** $j \leftarrow n$ **to** 1 **do****13 |** $B[C[A[j]]] \leftarrow A[j]$ **14 |** $C[A[j]] \leftarrow C[A[j]] - 1$ **15 end****16 end**

A →

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

 $k = 5$

C →

0	1	2	3	4	5
0	2	3	4	7	7

B →

1	2	3	4	5	6	7	8
0	0		2	3	3	3	5

1 Algorithm: COUNTING_SORT

Input: int* A, int n, int k**2 begin****3 for** $i \leftarrow 0$ **to** k **do****4 |** $C[i] \leftarrow 0$ **5 end****6 for** $j \leftarrow 1$ **to** n **do****7 |** $C[A[j]] \leftarrow C[A[j]] + 1$ **8 end****9 for** $i \leftarrow 1$ **to** k **do****10 |** $C[i] \leftarrow C[i] + C[i - 1]$ **11 end****12 for** $j \leftarrow n$ **to** 1 **do****13 |** $B[C[A[j]]] \leftarrow A[j]$ **14 |** $C[A[j]] \leftarrow C[A[j]] - 1$ **15 end****16 end**

A →

1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

 $k = 5$

C →

0	1	2	3	4	5
0	2	2	4	7	7

B →

1	2	3	4	5	6	7	8
0	0	2	2	3	3	3	5

Considerações sobre o método *CountingSort*

- ▶ O tempo de execução é dado em função do valor de k .
- ▶ Roda em tempo $\Theta(n + k)$.
- ▶ Se tivermos $k = O(n)$, então o algoritmo executa em tempo $\Theta(n)$.
- ▶ Exemplo prático de uso: vídeo locadora.

Radix Sort

Definição

Pressupõe que as chaves de entrada possuem limite no valor e no tamanho (quantidade de dígitos).

Ideia básica:

- ▶ Ordena em função dos dígitos (um de cada vez):
 - ▶ A partir do mais significativo?
 - ▶ Ou a partir do menos significativo?
- ▶ É essencial utilizar um segundo algoritmo estável para realizar a ordenação de cada dígito.

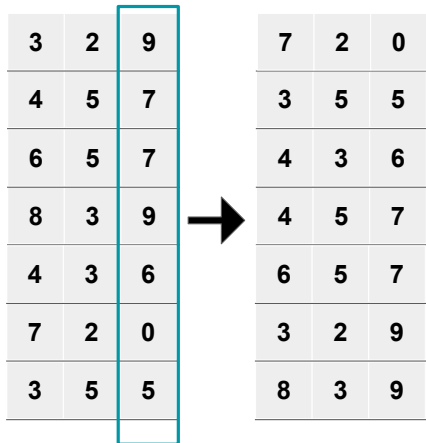
Funcionamento

A partir dos dígitos menos significativos:

3	2	9
4	5	7
6	5	7
8	3	9
4	3	6
7	2	0
3	5	5

Funcionamento

A partir dos dígitos menos significativos:



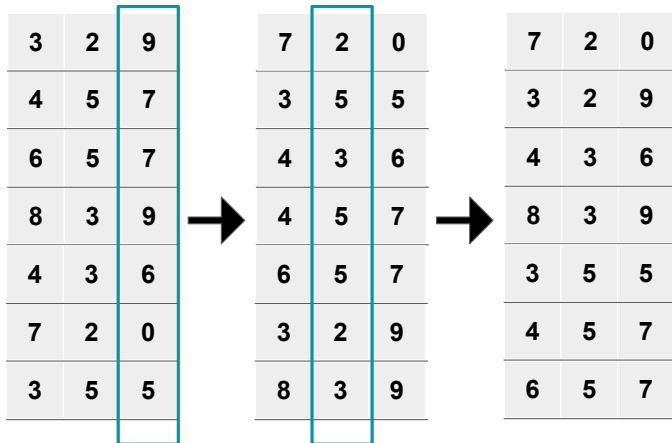
Funcionamento

A partir dos dígitos menos significativos:

3	2	9	→	7	2	0
4	5	7		3	5	5
6	5	7		4	3	6
8	3	9		4	5	7
4	3	6		6	5	7
7	2	0		3	2	9
3	5	5		8	3	9

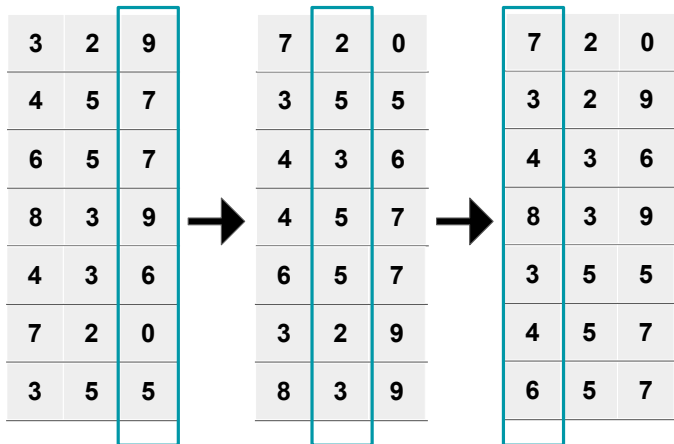
Funcionamento

A partir dos dígitos menos significativos:



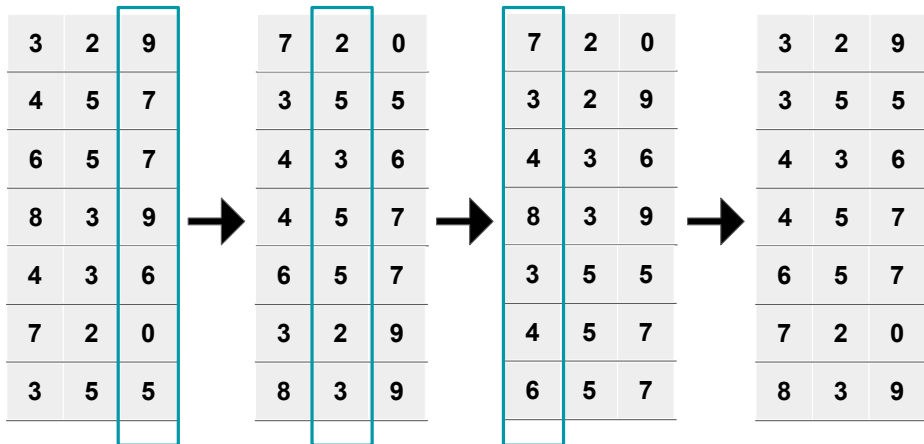
Funcionamento

A partir dos dígitos menos significativos:



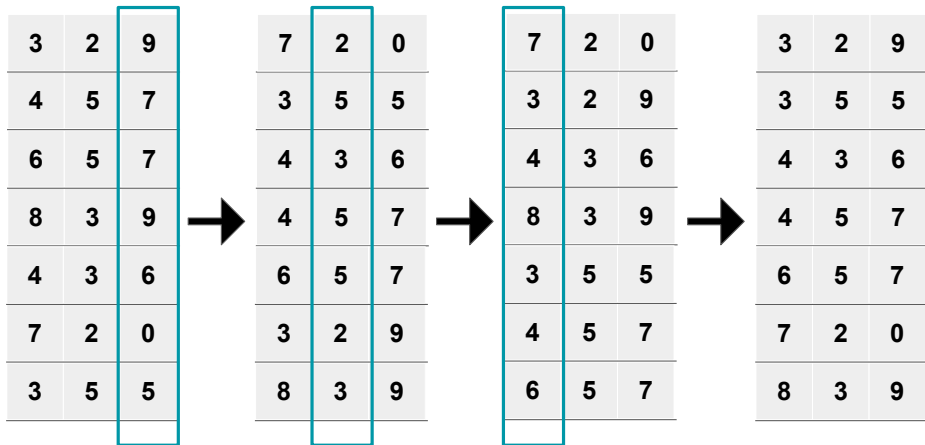
Funcionamento

A partir dos dígitos menos significativos:



Funcionamento

A partir dos dígitos menos significativos: (Como ficaria a partir do dígito mais significativo?)



Radix Sort - Pseudo Código

- ▶ Como dito anteriormente, o *Radix Sort* consiste em usar um outro método de ordenação (estável) para ordenar as chaves em relação a cada dígito.
- ▶ O código, portanto, é muito simples:

```
1 Algorithm: RADIX_SORT
```

```
   Input: int* v, int n, int d
```

```
2 begin
```

```
3   for  $i \leftarrow 0$  to  $d$  do
```

```
4     // utilizar um algoritmo estável para ordenar o array v pelo i-ésimo dígito
```

```
5     ORDENA_I_ESIMO_DIGITO(v, i, n)
```

```
6   end
```

```
7 end
```

- ▶ Onde:
 - ▶ d é número de dígitos.
 - ▶ v é o array de entrada.
 - ▶ n é o tamanho do array de entrada.

Bucket Sort

Definição

Pressupõe que a entrada consiste em elementos distribuídos de forma uniforme sobre o intervalo $[0, 1)$.

Ideia básica:

- ▶ A ideia do *Bucket Sort* é dividir o intervalo $[0, 1)$ em n subintervalos de mesmo tamanho (baldes), e então distribuir os n números nos baldes.
- ▶ Uma vez que as entradas são uniformemente distribuídas não se espera que muitos números caiam em cada balde.



Conceitos do *Bucket Sort*

Para produzir a saída ordenada, basta ordenar os números em cada balde, e depois examinar os baldes em ordem, listando seus elementos.

A função para determinação do índice do balde correto é $\lfloor n \times A[i] \rfloor$.

Vamos a um exemplo com 10 números:

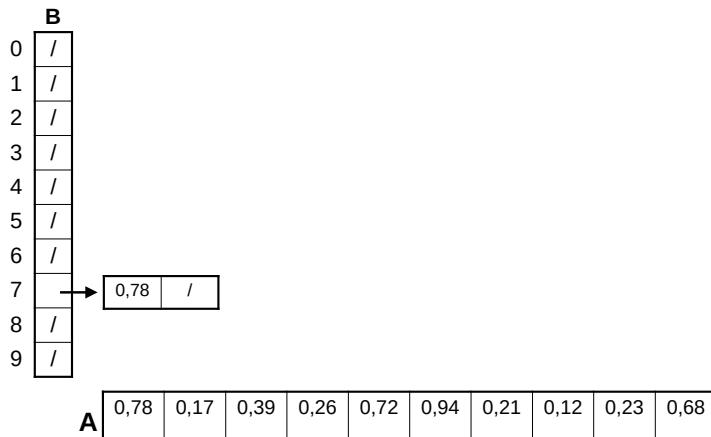
- ▶ A é o array de entrada.
- ▶ B é o array com os baldes.

Funcionamento

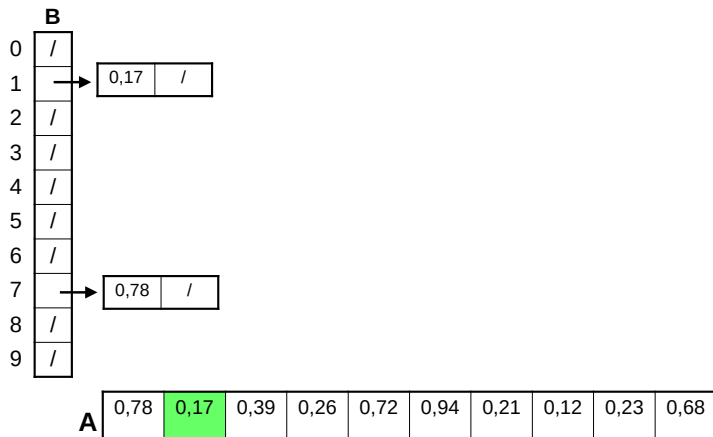
B	
0	/
1	/
2	/
3	/
4	/
5	/
6	/
7	/
8	/
9	/

A	0,78	0,17	0,39	0,26	0,72	0,94	0,21	0,12	0,23	0,68

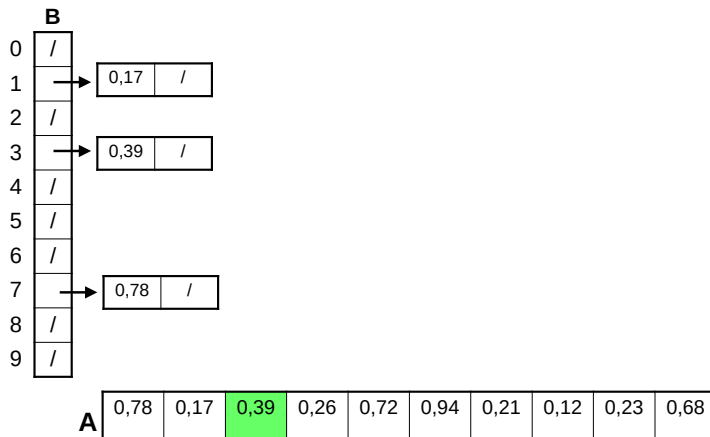
Funcionamento



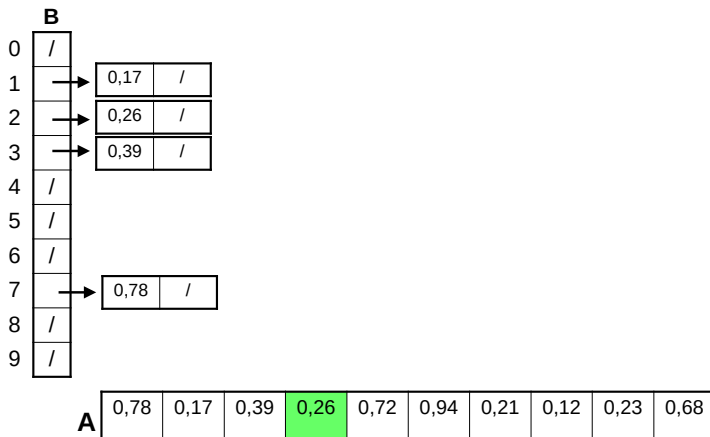
Funcionamento



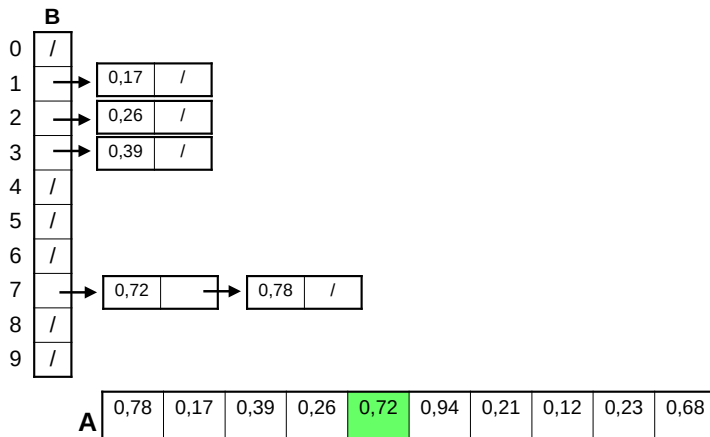
Funcionamento



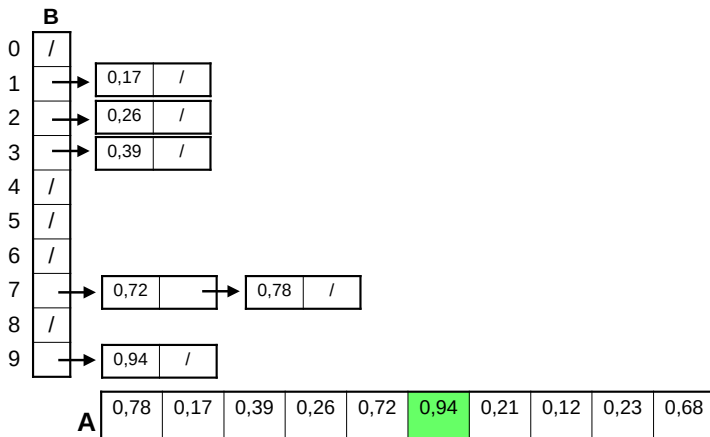
Funcionamento



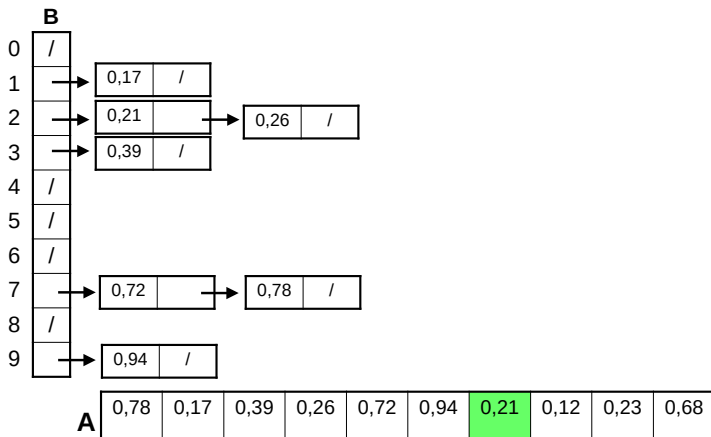
Funcionamento



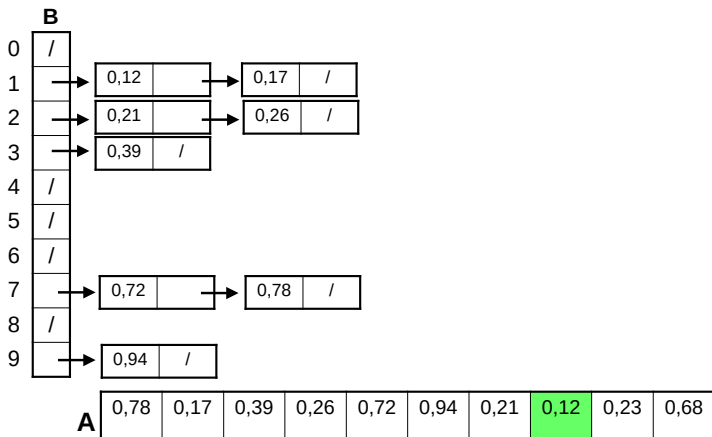
Funcionamento



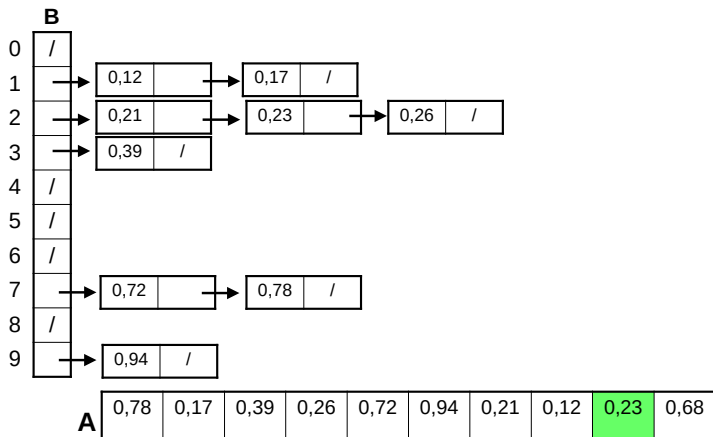
Funcionamento



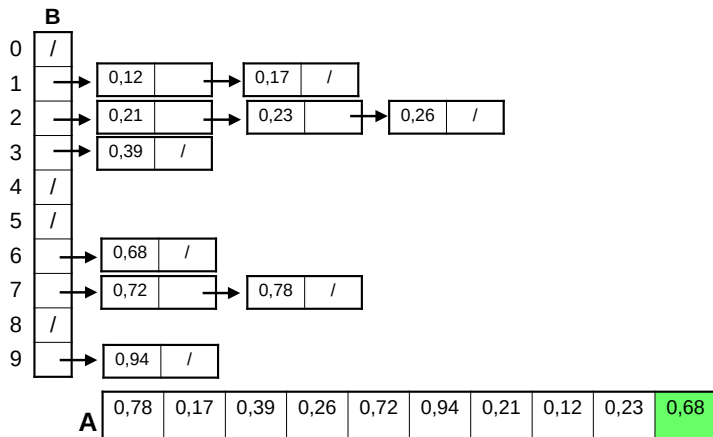
Funcionamento



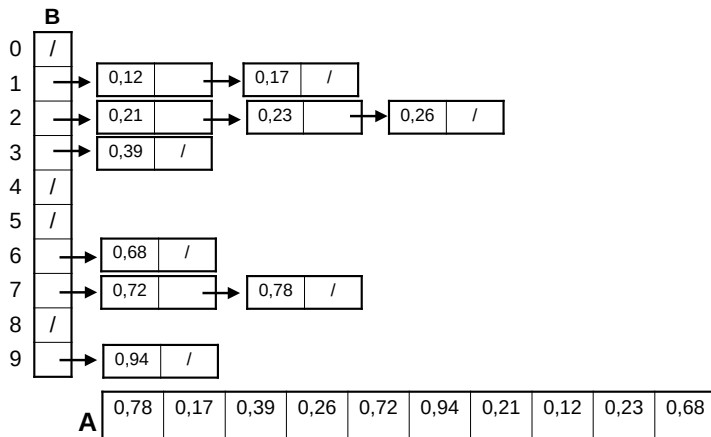
Funcionamento



Funcionamento



Funcionamento



Considerações Finais

Conclusão

- ▶ Foram vistos três algoritmos de ordenação linear (tempo $\Theta(n)$). Que são então melhores que os algoritmos de ordenação por comparação (tempo $O(n \lg n)$).
- ▶ Entretanto, nem sempre é interessante utilizar um destes três algoritmos:
 - ▶ Todos eles pressupõem algo sobre os dados de entrada a serem ordenados.

Pesquisa Sequencial e Pesquisa Binária

Bibliografia

Bibliografia

Os conteúdos deste material, incluindo figuras, textos e códigos, foram extraídos ou adaptados de:



Fernandes, Cristina G.

MAC0338 - Ordenação em tempo linear.

https://www.ime.usp.br/~cris/aulas/11_1_338/slides/aula10.pdf.

Acessado em 2021.



Cormen, Thomas H. and Leiserson, Charles E. and Rivest, Ronald L. and Stein, Clifford.

Introduction to Algorithms.

The MIT Press, 2011.

Exercício

- ▶ Dada a sequência de números: 3 4 0 2 5 1 3.
- ▶ Ordene em ordem crescente utilizando o algoritmo aprendido em sala (**Counting Sort**), apresentando a sequência dos números a cada passo (Teste de Mesa).